Product Requirements Document (PRD)

SafeMAPS — Public AI + MCP Interactive Demo

Product: SafeMAPS
Feature: Hosted AI Chat + Live MCP Tool-Call Demo
Version: 1.0
Status: Proposed
Primary Goal: Turn SafeMAPS from an MCP provider that requires manual setup into a publicly accessible, zero-setup demonstration of an AI agent consuming SafeMAPS MCP tools.

---

1. Executive Summary

SafeMAPS currently exposes its safety, routing, AQI, and geospatial capabilities through an MCP server. However, demonstrating those capabilities requires users to configure an MCP-compatible host and connect it to the SafeMAPS MCP endpoint.

This creates friction for recruiters, developers, and other visitors evaluating the project.

The proposed feature introduces a publicly hosted **SafeMAPS AI Demo** where a visitor can open a web page and ask natural-language questions such as:

> "What is the safest route from Koramangala to Whitefield?"

The application will use an LLM such as Claude for reasoning and tool selection while connecting to the existing SafeMAPS MCP server as an MCP client.

The interface will expose the agent's tool activity in real time.

For example:

```
User:
Compare the safest and fastest routes from
Koramangala to Whitefield.

SafeMAPS AI:
Understanding request...

🔧 get_safe_route
   profile: safest
   origin: Koramangala
   destination: Whitefield
```

```
✓ Tool completed

🔧 get_safe_route
   profile: fastest

✓ Tool completed

🔧 compare_route_profiles

✓ Tool completed

The safest route takes approximately 8 minutes
longer but reduces accident-risk exposure by 27%.
```

The objective is not to replace the existing SafeMAPS map application.

Instead, the AI Demo becomes an additional interface demonstrating:

- MCP
- AI agent orchestration
- LLM tool calling
- geospatial intelligence
- SafeMAPS routing
- real-time streaming
- full-stack AI engineering

---

2. Problem Statement

SafeMAPS currently has an MCP server but does not provide a zero-setup way for someone to experience it.

A visitor currently needs to:

1. Discover the repository.
2. Read MCP configuration instructions.
3. Configure an MCP-compatible client.
4. Connect the SafeMAPS MCP endpoint.
5. Send prompts.
6. Inspect the response.

This is acceptable for developers integrating SafeMAPS but poor for demonstrating the project.

For portfolio visitors and recruiters, the desired experience is:

```
README
   ↓
"Try Live AI Demo"
   ↓
Browser
   ↓
Ask question
   ↓
Watch MCP tools execute
   ↓
See SafeMAPS result
```

No installation or MCP configuration should be required.

---

3. Product Vision

SafeMAPS should demonstrate not only:

> "I built an MCP server."

but:

> "I built an AI application where an LLM dynamically discovers and invokes geospatial tools through MCP."

The AI Demo should make the architecture observable.

Instead of hiding tool execution, the UI intentionally exposes it.

This makes the application simultaneously:

1. A usable SafeMAPS conversational interface.
2. A visual demonstration of MCP.
3. A portfolio demonstration of AI-agent engineering.

---

4. Goals

4.1 Primary Goal

Create a publicly accessible conversational AI interface capable of using SafeMAPS MCP tools dynamically.

4.2 Functional Goals

The system must allow users to:

- Ask natural-language SafeMAPS questions.
- Ask route-related questions.
- Request safest/fastest/balanced routes.
- Compare routes.
- Query AQI information.
- Query nearby accident/safety information.
- Ask follow-up questions.
- See MCP tool calls while they execute.
- See structured tool inputs.
- See tool completion/failure state.
- Receive a final natural-language response.
- Optionally visualize resulting routes on a map.

4.3 Engineering Goals

The project should demonstrate:

- MCP client implementation
- MCP server implementation
- LLM integration
- agent/tool orchestration
- streaming responses
- FastAPI
- React
- asynchronous APIs
- geospatial systems
- production API security
- deployment

---

5. Non-Goals

Version 1 should NOT attempt to build:

- a general-purpose AI assistant
- autonomous multi-agent workflows
- voice interaction
- mobile applications
- user accounts
- long-term conversation memory

- trip history
- personalized recommendations
- autonomous navigation
- production navigation guidance

The first release should remain focused on demonstrating SafeMAPS + MCP extremely well.

---

6. Target Users

User Type 1 — Recruiter / Hiring Manager

Goal:

Understand the technical sophistication of SafeMAPS within 1–2 minutes.

Expected behavior:

1. Opens README.
2. Clicks "Live AI Demo."
3. Clicks a sample prompt.
4. Watches MCP tools execute.
5. Sees route/map result.

User Type 2 — Developer

Goal:

Understand how SafeMAPS implements MCP.

Expected behavior:

- inspect tool names
- inspect tool parameters
- inspect responses
- understand the client/server relationship
- inspect GitHub architecture afterward

User Type 3 — General Visitor

Goal:

Experiment with SafeMAPS through natural language without understanding MCP.

---

7. Core User Stories

US-01

As a visitor, I want to ask:

> "Safest route from Koramangala to Whitefield"

so that I can receive a safety-aware route without manually interacting with APIs.

US-02

As a visitor, I want to see which MCP tools the AI uses so that I understand how the result was produced.

US-03

As a visitor, I want to compare safest and fastest routes so that I understand the trade-off between safety and travel time.

US-04

As a visitor, I want to ask:

> "What's the AQI near Indiranagar?"

so that the AI can query SafeMAPS environmental information.

US-05

As a developer, I want to inspect MCP tool arguments so that I can understand how the agent communicates with SafeMAPS.

US-06

As a recruiter, I want the demo to work without installing anything.

---

8. User Experience

8.1 Entry Page

Recommended route:

```
/ai
```

or

```
/demo
```

Header:

```
SafeMAPS AI

AI-powered safer routing using
Model Context Protocol
```

Secondary text:

```
Ask SafeMAPS about routes, safety,
accidents and air quality.
```

---

9. Suggested Prompts

Before the first message, show clickable examples.

Examples:

```
Safest route from Koramangala to Whitefield

Compare safest vs fastest route from
Indiranagar to Electronic City

What's the AQI around MG Road?
```

```
Which route has lower accident exposure?
```

Clicking a suggestion should immediately submit the prompt.

---

10. Main Interface

Recommended desktop layout:

```
┌──────────────────────────────────────────────────────────┐
│ SafeMAPS AI                              MCP Connected • │
├──────────────────────────┬───────────────────────────────┤
│                          │                               │
│ CHAT                     │ MAP                           │
│                          │                               │
│ User messages            │ Route visualization           │
│                          │                               │
│ AI responses             │ Safety overlays               │
│                          │                               │
│ MCP tool cards           │ Accident markers              │
│                          │                               │
│                          │ AQI markers                   │
│                          │                               │
├──────────────────────────┴───────────────────────────────┤
│ Ask SafeMAPS...                                Send →    │
└──────────────────────────────────────────────────────────┘
```

Recommended ratio:

```
Chat: 40–45%
Map: 55–60%
```

Mobile layout:

```
Chat
↓
Map
↓
Input
```

---

11. Tool Call Visualization

This is one of the highest-priority requirements.

Every MCP tool execution should appear as a visible card.

Example loading state:

```
┌──────────────────────────────────┐
│ 🔧 MCP TOOL                      │
│                                  │
│ get_safe_route                   │
│                                  │
│ origin      Koramangala          │
│ destination Whitefield           │
│ profile     safest               │
│                                  │
│ ○ Running...                     │
└──────────────────────────────────┘
```

Success:

```
┌──────────────────────────────────┐
│ ✓ MCP TOOL                       │
│                                  │
│ get_safe_route                   │
│                                  │
│ Completed in 742 ms              │
│                                  │
│ View result ▾                    │
└──────────────────────────────────┘
```

Failure:

```
┌──────────────────────────────────┐
│ ⚠ MCP TOOL                       │
│                                  │
│ get_safe_route                   │
│                                  │
│ Tool execution failed            │
│                                  │
```

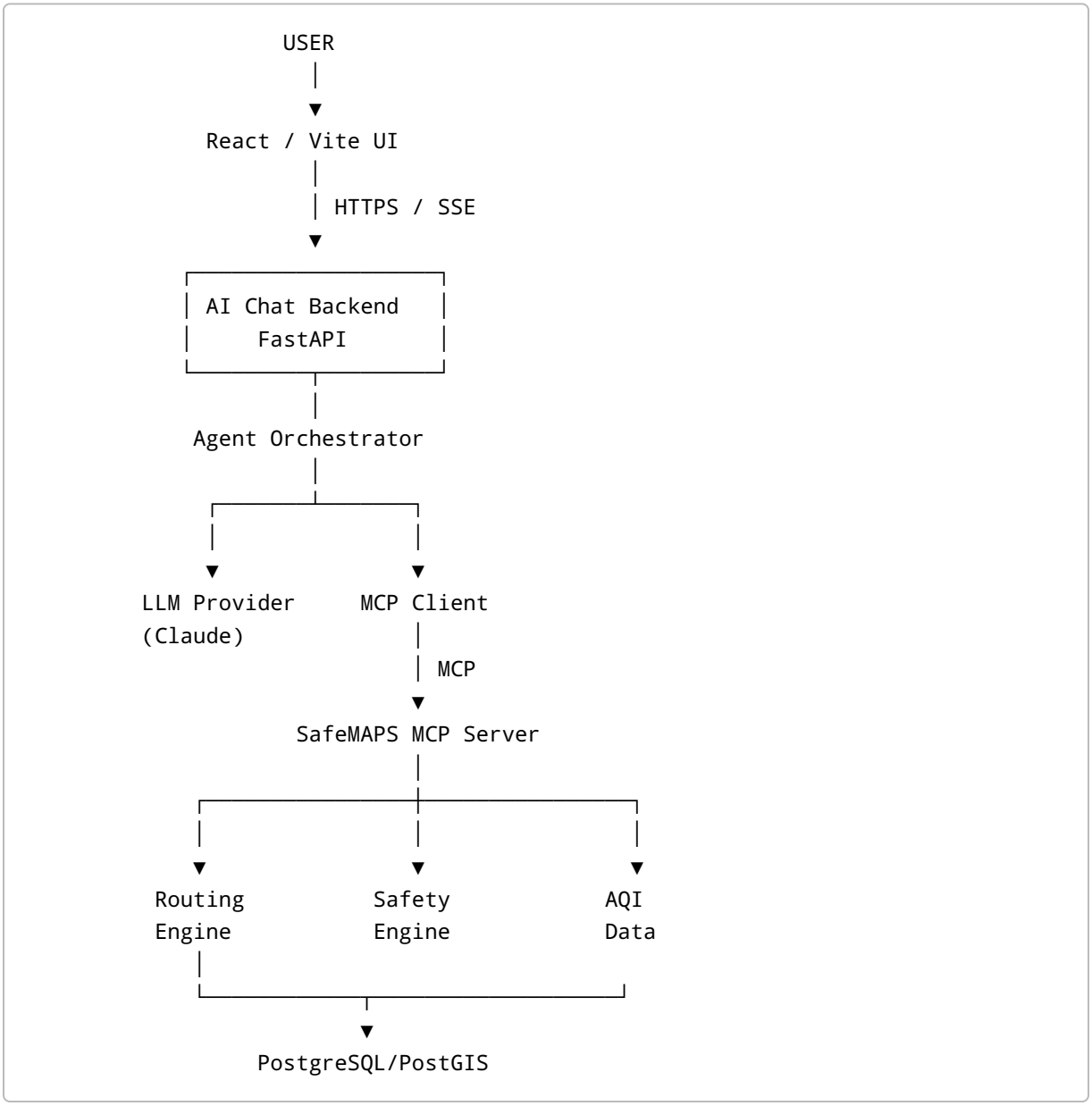
```
| Retry                              |
└────────────────────────────────────┘
```

Tool results should be collapsible to prevent overwhelming nontechnical users.

---

12. Architecture

Recommended production architecture:



---

13. Security Architecture

The browser MUST NOT contain the Anthropic API key.

Incorrect:

```
Browser
   ↓
Anthropic API
```

if the API credential is shipped with client-side JavaScript.

Recommended:

```
Browser
   ↓
SafeMAPS AI Backend
   ↓
Anthropic API
```

Environment variables:

```
ANTHROPIC_API_KEY=
MCP_SERVER_URL=
DATABASE_URL=
ALLOWED_ORIGINS=
```

Secrets must never be:

- committed to Git
- bundled into Vite
- returned through APIs
- exposed through browser developer tools

---

14. Backend Components

Recommended new structure:

```
backend/
│
├── main.py
├── mcp_server.py
│
├── ai/
│   ├── __init__.py
│   ├── agent.py
│   ├── anthropic_client.py
│   ├── mcp_client.py
│   ├── session.py
│   └── prompts.py
│
├── api/
│   └── chat.py
│
└── services/
```

---

15. MCP Client

Create:

```
backend/ai/mcp_client.py
```

Responsibilities:

- connect to SafeMAPS MCP endpoint
- initialize MCP session
- discover available tools
- expose tools to the agent
- invoke tools
- capture results
- capture errors
- capture execution duration

Important principle:

Do NOT manually duplicate every MCP tool definition inside the AI application if MCP discovery can supply them.

Desired workflow:

```
connect()
   ↓
initialize()
   ↓
list_tools()
   ↓
convert MCP schemas → LLM tool schemas
   ↓
LLM chooses tool
   ↓
call_tool()
```

This is important because it demonstrates actual MCP usage rather than ordinary hard-coded function calling.

---

16. Agent Orchestrator

Create:

```
backend/ai/agent.py
```

The agent controls the reasoning/tool loop.

Conceptual workflow:

```
USER MESSAGE
      │
      ▼
LLM
      │
      ├── Final answer?
      │       ↓
      │     return
      │
      └── Tool request?
              │
              ▼
          MCP Client
              │
              ▼
          MCP Server
```

```
            │
            ▼
        Tool Result
            │
            ▼
           LLM
            │
            └── repeat if required
```

The loop should have a maximum number of tool iterations.

Recommended:

```
MAX_TOOL_ITERATIONS = 6
```

This protects against runaway requests and unnecessary API costs.

---

17. System Prompt

The SafeMAPS agent should receive a dedicated system prompt.

Example behavioral requirements:

```
You are SafeMAPS AI, an assistant specialized
in safer urban routing.

Use SafeMAPS MCP tools whenever information
about routes, safety, accidents or AQI is required.

Never invent route statistics.

Never invent accident statistics.

Never invent AQI values.

When comparing routes, clearly communicate
trade-offs between:

- travel time
- distance
- safety exposure
- accident exposure
```

```
- air-quality exposure

SafeMAPS is an informational demonstration
and should not be represented as guaranteed
navigation or personal-safety advice.
```

---

18. Chat API

Recommended endpoint:

```
POST /api/ai/chat
```

Request:

```
{
  "message": "Safest route from Koramangala to Whitefield",
  "session_id": "..."
}
```

Because tool activity needs to appear live, streaming is strongly recommended.

Potential technologies:

- Server-Sent Events
- streaming HTTP
- WebSocket

For V1, Server-Sent Events or streaming HTTP is sufficient.

---

19. Streaming Event Protocol

Backend should emit structured events.

Assistant started

```
{
  "type": "assistant_start"
}
```

Tool started

```
{
  "type": "tool_start",
  "tool": "get_safe_route",
  "arguments": {
    "origin": "Koramangala",
    "destination": "Whitefield",
    "profile": "safest"
  }
}
```

Tool finished

```
{
  "type": "tool_result",
  "tool": "get_safe_route",
  "duration_ms": 742,
  "result": {}
}
```

Text

```
{
  "type": "text_delta",
  "content": "The safest route..."
}
```

Finished

```
{
  "type": "done"
}
```

Error

```
{
  "type": "error",
  "message": "Unable to retrieve route."
}
```

---

20. Frontend Components

Recommended structure:

```
src/
│
├── pages/
│   └── AIDemo.jsx
│
├── components/
│   └── ai/
│       ├── ChatPanel.jsx
│       ├── ChatInput.jsx
│       ├── UserMessage.jsx
│       ├── AssistantMessage.jsx
│       ├── ToolCallCard.jsx
│       ├── ToolResult.jsx
│       ├── SuggestedPrompts.jsx
│       ├── MCPStatus.jsx
│       └── AIMap.jsx
│
└── services/
    └── chatApi.js
```

---

21. Map Integration

Route-producing MCP tools should return machine-readable geometry.

Preferred representation:

```
GeoJSON
```

When:

```
get_safe_route
```

returns a route, the frontend should automatically render it.

Example:

```
AI asks MCP
     ↓
get_safe_route
     ↓
GeoJSON
     ↓
stream event
     ↓
React
     ↓
MapLibre / Leaflet
     ↓
route displayed
```

---

22. Route Comparison Visualization

When comparing routes, display both visually.

Example:

```
SAFEST

18.7 km
46 min
Safety score: 82

FASTEST

16.9 km
39 min
Safety score: 63
```

Then AI explanation:

```
The safest route adds approximately seven
minutes but has substantially lower modeled
accident-risk exposure.
```

Do not present modeled risk scores as guarantees of actual personal safety.

---

23. MCP Connection Indicator

Header should expose system status:

```
● MCP Connected
```

Possible states:

```
● Connected

○ Connecting

● Unavailable
```

This is valuable because MCP itself is part of the project's demonstration.

---

24. Tool Discovery Panel

Optional but highly valuable.

Add:

```
MCP Tools
7 available
```

Clicking opens:

```
SafeMAPS MCP Tools

✓ get_safe_route
✓ compare_route_profiles
✓ get_aqi_near
✓ get_accident_risk_near
✓ ...
```

This reinforces that tools are discovered through MCP.

---

25. Conversation State

V1 should maintain temporary session history.

Example:

```
User:
Safest route from Koramangala to Whitefield

AI:
...

User:
What about the fastest one?
```

The second question should retain:

```
origin = Koramangala
destination = Whitefield
```

Long-term persistence is unnecessary.

State may initially live:

- in frontend memory, or
- in temporary server session storage.

---

26. Loading States

The application must avoid showing a generic spinner for the entire operation.

Instead expose progress:

```
Understanding request...

Connecting to SafeMAPS MCP...

Calling get_safe_route...

Analyzing route...
```

```
Generating response...
```

This creates the visual experience of an agent using tools.

---

27. Error Handling

MCP unavailable

```
SafeMAPS MCP is temporarily unavailable.

Please try again shortly.
```

LLM provider unavailable

```
The AI service is temporarily unavailable.
```

Tool failure

Show the failed tool card and allow the agent to continue if possible.

Invalid location

```
I couldn't identify that location.

Try specifying a more precise Bengaluru locality.
```

No route

```
SafeMAPS couldn't calculate a route between
those locations.
```

---

28. Rate Limiting

A public LLM endpoint can generate unexpected API costs.

Implement rate limiting.

Initial recommendation:

```
10 AI requests / IP / 10 minutes
```

and:

```
50 requests / IP / day
```

Exact limits can be adjusted after observing usage.

Also enforce:

```
MAX_TOOL_ITERATIONS
MAX_MESSAGE_LENGTH
REQUEST_TIMEOUT
```

---

29. Abuse Protection

Protect against:

- prompt flooding
- extremely large messages
- automated API abuse
- repeated expensive routing calls
- tool-loop attacks

Potential controls:

```
rate limiting
input size limits
timeouts
tool iteration limits
LLM token limits
CORS restrictions
```

---

30. Observability

Record non-sensitive operational metrics.

Useful metrics:

```
request_id
timestamp
LLM latency
MCP latency
tool name
tool execution duration
number of tool calls
response duration
success/failure
```

Never log:

```
API keys
authorization headers
sensitive environment variables
```

---

31. Performance Targets

Target perceived latency:

Initial feedback:

```
< 500 ms
```

Tool-call visualization:

```
< 2 seconds
```

Typical complete answer:

```
3–8 seconds
```

Longer operations should continuously stream status events.

---

32. Deployment Architecture

Recommended:

```
GitHub
   │
   ├── frontend
   │       ↓
   │    Vercel / similar
   │
   └── backend
           ↓
      Render / Railway / similar
           │
           ├── AI API
           ├── MCP Client
           └── MCP Server
                  │
                  ▼
              PostgreSQL
```

The AI backend and MCP server can initially run in the same deployed backend application while remaining logically separate.

---

33. README Integration

The GitHub README should prominently contain:

```
## 🚀 Live AI + MCP Demo

Try SafeMAPS without installing anything.

Ask:

"Compare the safest and fastest routes from
Koramangala to Whitefield."

[Launch SafeMAPS AI]
```

Below it:

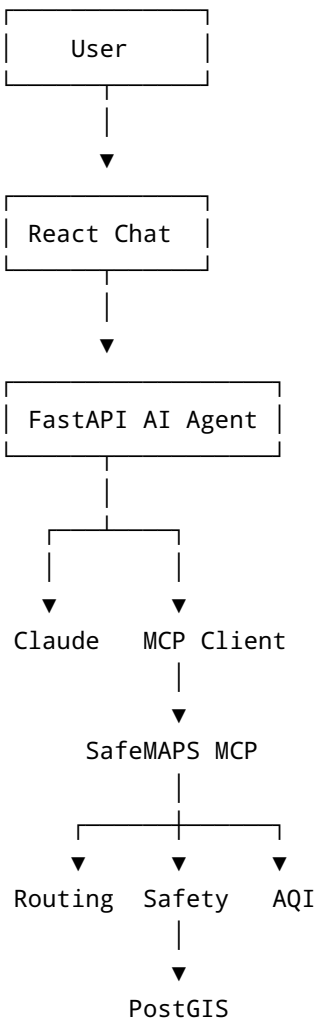
```
The demo uses Claude as the reasoning model
and communicates with SafeMAPS capabilities
through Model Context Protocol (MCP).

Tool calls are displayed live so you can see
how the agent interacts with SafeMAPS.
```

---

34. Architecture Diagram for README

Recommended:



35. MVP Requirements

The first deployable version should contain only the functionality necessary to prove the architecture.

P0 — Mandatory

- `/ai` page
- conversational input
- LLM integration
- server-side API key
- MCP client
- MCP connection
- MCP tool discovery
- dynamic tool invocation
- live tool-call cards
- streamed final answer
- error handling
- basic rate limiting
- suggested prompts

P1 — Strongly Recommended

- route map
- GeoJSON rendering
- safest/fastest route comparison
- MCP connection status
- expandable tool results
- conversation context
- mobile responsiveness

P2 — Future

- animated route rendering
- AQI overlays
- accident heatmap
- conversation sharing
- demo replay
- tool execution timeline
- analytics dashboard
- multiple model providers

36. Recommended Development Phases

Phase 1 — MCP Client

Build:

```
backend/ai/mcp_client.py
```

Success criterion:

The backend can connect to the existing SafeMAPS MCP server, call `list_tools()`, invoke one discovered tool, and print the result.

Do this before building the chat UI.

Phase 2 — LLM Tool Loop

Build:

```
anthropic_client.py
agent.py
```

Success criterion:

Terminal interaction:

```
> safest route from Koramangala to Whitefield

LLM
→ selects get_safe_route

MCP
→ executes tool

LLM
→ explains result
```

No frontend required yet.

Phase 3 — Chat API

Implement:

```
POST /api/ai/chat
```

Success criterion:

A request from Postman/curl can execute the complete LLM → MCP → LLM loop.

Phase 4 — Streaming

Add:

```
tool_start
tool_result
text_delta
done
```

Success criterion:

Tool calls appear before final generation completes.

Phase 5 — React Chat UI

Implement:

```
AIDemo
ChatPanel
ChatInput
AssistantMessage
ToolCallCard
```

Success criterion:

Browser users can conduct a complete SafeMAPS conversation.

Phase 6 — Map

Connect route geometry from MCP responses to the existing mapping stack.

Success criterion:

```
User prompt
      ↓
MCP route
      ↓
map automatically updates
```

Phase 7 — Production Hardening

Add:

- rate limiting
- timeout handling
- input limits
- API error handling
- logging
- CORS configuration
- environment configuration
- deployment health checks

Phase 8 — Public Demo

Deploy and add:

```
LIVE DEMO
```

to GitHub README.

---

37. Acceptance Criteria

The feature is considered complete when a first-time visitor can:

1. Open the public SafeMAPS AI URL.
2. Enter:

"Compare the safest and fastest routes from Koramangala to Whitefield."

1. See the AI initiate MCP tool calls.
2. See the names of those MCP tools.
3. See their running/completed states.
4. Receive actual SafeMAPS results.
5. See the resulting route(s) on the map.
6. Receive a natural-language comparison.
7. Ask a follow-up question.
8. Complete all of this without installing or configuring anything.

---

38. Critical Technical Requirement

The project must use MCP as the actual integration boundary.

Avoid implementing:

```
Claude
   ↓
hardcoded Python functions
   ↓
SafeMAPS
```

while merely displaying "MCP Tool" in the frontend.

The intended architecture is:

```
Claude
   ↓
tool request
   ↓
MCP CLIENT
   ↓
MCP protocol
   ↓
SafeMAPS MCP SERVER
   ↓
SafeMAPS services
```

This distinction is important because it is what makes the project a genuine MCP client + server implementation.

---

39. Definition of Success

The project succeeds when a recruiter or developer can understand within approximately 60 seconds that:

> SafeMAPS is an AI-powered geospatial system where an LLM dynamically uses safety, routing, accident, and environmental capabilities exposed through Model Context Protocol.

The visitor should not need to read the source code to understand this.

They should be able to **watch the architecture work.**

---

40. Final Product Experience

The ideal final experience is:

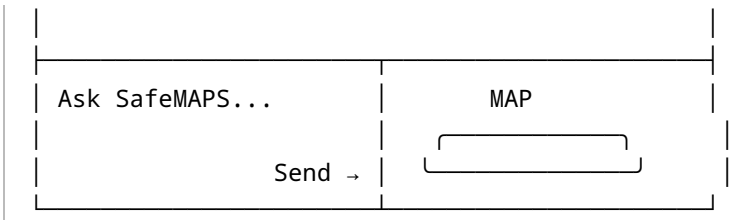
```
GitHub README

       ↓

🚀 TRY LIVE MCP DEMO

       ↓

┌──────────────────────────────────────────────┐
│ SafeMAPS AI                 ● MCP Connected  │
│                                              │
│ You                                          │
│ Safest route from Koramangala to Whitefield  │
│                                              │
│ SafeMAPS AI                                  │
│                                              │
│ 🔧 get_safe_route                            │
│    profile: safest                           │
│    ○ Running...                              │
│                                              │
│ ✓ get_safe_route                  742 ms     │
│                                              │
│ 🔧 get_accident_risk_near                    │
│    ○ Running...                              │
│                                              │
│ ✓ get_accident_risk_near          381 ms     │
│                                              │
│ The safer route takes approximately...       │
```



The central portfolio message becomes:

"I didn't just expose tools through MCP. I built and deployed the complete MCP-powered AI application that consumes them."